Estudio comparativo del rendimiento de un árbol de decisión C4.5 propio y del modelo de ensamble Random Forest en problemas de clasificación

Laura Cornaló Bassan[1 [00000]], Matías Fernandez[1 [24282]], Mariano Insaurralde[1 [24684]], Philippe Maurel Garcete[1 [25613]], and Franco Sanchez[1 [27310]]

Universidad Tecnológica Nacional, Facultad Regional de Resistencia
Ingeniería en Sistemas de Información
{correo1,correo2,correo3,correo4,correo5}@gmail.com

Abstract. Keywords: Decision Tree · C4.5 · Random Forest

1 Introducción

Un clasificador automático asigna una clase a cada caso a partir de sus características. Los árboles de decisión resuelven esta tarea mediante una secuencia de preguntas sobre esas características y producen un modelo que una persona puede leer. C4.5 es uno de los algoritmos más difundidos de la familia: elige cada pregunta con un criterio orientado a separar las clases, acepta atributos numéricos y categóricos, tolera datos faltantes e incorpora mecanismos para frenar el crecimiento del árbol y evitar que se ajuste en exceso al conjunto de entrenamiento [2]. Cada camino de la raíz a una hoja equivale a una regla del tipo "si se cumple tal condición, se predice tal clase", de modo que las decisiones del modelo pueden explicarse y revisarse sin herramientas adicionales.

Un árbol aislado es sensible a pequeños cambios en los datos: agregar o quitar unos pocos casos de entrenamiento puede dar una estructura distinta. Random Forest (bosque aleatorio) atenúa este problema entrenando muchos árboles y combinando sus respuestas. Cada árbol se construye con una muestra aleatoria de los casos y, en cada división, evalúa solo un subconjunto al azar de las características; la clase final se obtiene combinando las respuestas de los árboles. Así se diluye el peso de los casos atípicos y de los atributos dominantes, y por lo general la precisión puede mejorar y las predicciones se vuelven más estables. A cambio, el cómputo es más costoso y una decisión individual deja de poder seguirse en una única regla, lo que explica que se lo describa como un modelo de caja negra [1].

Ninguno de los dos enfoques domina al otro en general. Si el problema es sencillo o hay que justificar cada decisión, un árbol puede bastar; si lo que importa es la precisión y no se exige explicar casos individuales, el bosque suele ser la opción más razonable. Ahora bien, la comparación solo es válida si ambos modelos se entrenan y prueban sobre las mismas particiones, se miden con las

mismas métricas y ninguno accede a información del conjunto de prueba durante el entrenamiento. Con esquemas de evaluación distintos, una diferencia de rendimiento puede reflejar la forma de medir y no el algoritmo.

Este trabajo realiza esa comparación sobre el conjunto de datos [DATASET PENDIENTE]. Se implementa desde cero un árbol de decisión basado en C4.5 y se lo contrasta con el Random Forest de scikit-learn [3]. Programar el árbol propio permite controlar y justificar cada decisión de construcción, mientras que el bosque de scikit-learn sirve de referencia de un método de ensamble de uso extendido. Los dos modelos se evalúan con las mismas particiones y métricas, y para cada uno se prueban varias configuraciones de parámetros con el fin de ver cómo responde su comportamiento.

El objetivo es determinar en qué condiciones conviene cada modelo, atendiendo a la precisión, pero también a la estabilidad de las predicciones, al tiempo de cómputo y a qué tan interpretable resulta cada uno. El diseño experimental busca que las diferencias observadas puedan atribuirse a los algoritmos y no a las condiciones de la prueba. De fondo está la idea de que la mayor complejidad de un bosque no garantiza una mejora: esta depende del problema, de la métrica que se priorice y del costo que se acepte pagar.

Las contribuciones son una implementación propia de C4.5, con decisiones explícitas sobre la construcción, el criterio de división y el control del crecimiento del árbol [2]; un esquema de evaluación común para ambos modelos; y un análisis comparativo de los cuatro aspectos mencionados, que puede orientar la elección entre árbol y bosque en problemas similares.

El resto del trabajo se organiza así: la sección 2 expone los fundamentos teóricos de C4.5 y Random Forest; la sección 3 describe brevemente los algoritmos desarrollados; la sección 4 presenta la solución implementada y los problemas encontrados; la sección 5 reúne las simulaciones y sus resultados; y la sección 6 discute las conclusiones. Las referencias cierran el documento.

2 Fundamentos teóricos

La introducción presentó los dos modelos y el problema de comparación que motiva el trabajo. En esta sección se fijan los conceptos y las fórmulas que usan las secciones siguientes, cada uno definido una sola vez. Se parte del criterio de construcción de un árbol de decisión, con el manejo de atributos continuos, valores faltantes y poda. Luego se expone la lógica del ensamble Random Forest y las nociones de margen, fuerza y cota que lo acompañan.

2.1 Árboles de decisión y C4.5

Un árbol de decisión clasifica un caso haciéndole preguntas sucesivas sobre sus atributos. Cada pregunta ocupa un nodo interno y reparte los casos en grupos; cada hoja guarda la clase que se asigna a los casos que llegan hasta ella. El camino de la raíz a una hoja es una regla, como se indicó en la introducción, de modo que el árbol completo equivale a un conjunto de reglas. Un caso de entrenamiento

con todos sus atributos conocidos recorre un único camino, así que, si no se podan las reglas, el árbol produce reglas mutuamente exclusivas y exhaustivas para esos casos [2]. Para decidir qué preguntar hace falta medir cuán mezcladas están las clases dentro de un nodo, magnitud que se llama impureza: un nodo puro contiene una sola clase y uno impuro reparte sus casos entre varias. C4.5 construye el árbol de forma recursiva y elige cada pregunta según esa medida.

La entropía mide la impureza de un conjunto de casos a partir de la distribución de sus clases. Sea D el conjunto de casos de un nodo, C el conjunto de clases y p_c la proporción de casos de la clase c, medida por masa cuando los pesos no son unitarios. La entropía, expresada en bits, está dada por la ecuación (1) [2].

$$H(D) = -\sum_{c \in C} p_c \log_2 p_c \tag{1}$$

Se adopta la convención $0 \log_2 0 = 0$ para que las clases ausentes no produzcan indeterminaciones. La entropía vale cero cuando todos los casos son de una misma clase y alcanza su máximo cuando las clases tienen proporciones iguales. Sobre esta medida se evalúa cada pregunta posible.

Una pregunta sobre un atributo A divide D en v subconjuntos D_j. Cada caso tiene un peso, que inicialmente es uno, y la masa de un conjunto es la suma de esos pesos: W_j es la masa de D_j y W la de D. La impureza que queda tras la división es el promedio de sus entropías, ponderado por la masa de cada subconjunto. Sin valores faltantes, la entropía de la partición resulta de la ecuación (2).

$$H_A(D) = \sum_{j=1}^{v} \frac{W_j}{W} H(D_j) \tag{2}$$

La ganancia de información es la reducción de entropía que produce la división, según la ecuación (3).

$$\mathrm{Gain}(D, A) = H(D) - H_A(D) \tag{3}$$

Cuanto mayor es la ganancia, más puras quedan las ramas, y una ganancia nula indica que la partición no reduce la impureza. Se mide en bits y no es negativa, porque la entropía de la partición es menor o igual que la del conjunto original. Sirve para comparar atributos, pero con un sesgo que corrige el paso siguiente.

La ganancia favorece a los atributos con muchos valores distintos: partir el conjunto en grupos muy pequeños reduce la impureza casi por construcción, aunque las reglas resultantes puedan generalizar mal [2]. C4.5 atenúa este efecto dividiendo la ganancia por la entropía de la propia partición, llamada información de la partición y dada por la ecuación (4).

$$\mathrm{SplitInfo}(D, A) = -\sum_{j=1}^{v} \frac{W_j}{W} \log_2 \frac{W_j}{W} \tag{4}$$

El cociente entre ambas magnitudes es la tasa de ganancia, ecuación (5) [2].

$$\mathrm{GainRatio}(D, A) = \frac{\mathrm{Gain}(D, A)}{\mathrm{SplitInfo}(D, A)} \tag{5}$$

Un atributo que fragmenta mucho el conjunto aumenta el denominador y queda penalizado frente a uno que lo fragmenta menos. Si la partición no produce al menos dos ramas conocidas de masa positiva, el atributo no se considera candidato; sin valores faltantes, eso equivale a un denominador nulo.

Los atributos numéricos no tienen un conjunto finito de categorías y requieren otro tratamiento. En cada nodo, C4.5 ordena los valores observados del atributo, operación de costo $O(|D| \log |D|)$, y examina únicamente los cortes entre valores consecutivos. Si hay r valores distintos, existen $r-1$ particiones binarias posibles, cada una definida por un umbral t que separa los casos con $A \leq t$ de los casos con $A > t$ [2]. Sin valores faltantes, para cada umbral se calcula la ganancia de la ecuación (6) y se conserva el corte con la mayor tasa de ganancia.

$$\mathrm{Gain}(D, A, t) = H(D) - \frac{W_{\leq t}}{W} H(D_{\leq t}) - \frac{W_{>t}}{W} H(D_{>t}) \tag{6}$$

El umbral coincide con un valor observado y se elige en cada nodo, de manera que no hay discretización global ni una grilla fijada de antemano. Geométricamente, cada condición $A \leq t$ define un semiespacio limitado por un hiperplano perpendicular al eje del atributo, y un árbol que usa solo atributos numéricos parte el espacio en regiones rectangulares alineadas con los ejes. Un atributo numérico puede reutilizarse más abajo, ya que los subconjuntos resultantes pueden admitir nuevos cortes, mientras que un atributo categórico se descarta una vez usado [2].

Los valores faltantes obligan a distinguir entre la masa total W de un nodo, la masa W_K de los casos que tienen valor para el atributo evaluado y la masa W_U de los que no lo tienen. La ganancia se calcula solo sobre el subconjunto D_K de casos conocidos y se escala por la fracción W_K/W, como indica la ecuación (7) [2].

$$\mathrm{Gain}^*(D, A) = \frac{W_K}{W} \left[H(D_K) - \sum_j \frac{W_j}{W_K} H(D_j) \right] \tag{7}$$

En ese cálculo, las proporciones de clase se obtienen con las masas y no con los conteos. En la información de la partición, los casos faltantes cuentan como una rama más dentro del cálculo, aunque no den lugar a una rama del árbol (ecuación (8)).

$$\begin{aligned} \mathrm{SplitInfo}^*(D, A) = & -\sum_j \frac{W_j}{W} \log_2 \frac{W_j}{W} \\ & - \frac{W_U}{W} \log_2 \frac{W_U}{W} \end{aligned} \tag{8}$$

En el entrenamiento, un caso con un valor faltante se envía a todas las ramas y su peso se reparte según las proporciones q_j de la ecuación (9).

$$q_j = \frac{W_j}{W_K} \tag{9}$$

En la predicción, cuando falta el atributo que decide el nodo, las probabilidades de las ramas para un caso x se combinan con esas mismas proporciones, según la ecuación (10).

$$P(c \mid x) = \sum_j q_j P(c \mid x, \text{rama } j) \tag{10}$$

La tasa de ganancia se obtiene con la ganancia y la información de la partición ya ajustadas. Con este reparto la masa total se conserva y no se imputa ningún valor.

Un árbol que crece sin restricciones puede terminar ajustándose a rasgos particulares del conjunto de entrenamiento y perder capacidad sobre casos nuevos, fenómeno conocido como sobreajuste. Cuanto más profundo es el árbol, menos casos respaldan cada hoja, y la tasa de acierto sobre el entrenamiento puede seguir subiendo sin que lo haga la tasa de acierto sobre datos nuevos. La poda limita ese riesgo de dos maneras. La prepoda detiene el crecimiento durante la construcción; la pospoda elimina subárboles una vez construido el árbol completo. La prepoda evita construir subárboles que luego se descartarían, mientras que la pospoda requiere construir el árbol completo antes de evaluarlo. El C4.5 original emplea pospoda pesimista, basada en una estimación del error, y la pospoda suele preferirse porque evalúa cada subárbol ya desarrollado [2].

El C4.5 original también aplica poda de reglas, que acorta las condiciones de cada regla y puede romper su exclusividad mutua y su carácter exhaustivo [2]. La prepoda tiene un inconveniente conocido como efecto de horizonte: puede frenar el crecimiento antes de tiempo, porque en ese momento no se ve lo que aportaría el subárbol completo. Hallar el árbol óptimo, pequeño y preciso a la vez, es un problema NP-completo [2]. No se conoce un algoritmo eficiente que lo resuelva en todos los casos, así que C4.5 elige en cada nodo la división de mayor tasa de ganancia y no vuelve a evaluar esa decisión durante la construcción; la pospoda actúa sobre los subárboles. Este trabajo adopta solamente prepoda y no implementa pospoda ni poda de reglas; las consecuencias de esa decisión se discuten en la sección 4.

Las definiciones anteriores establecen con qué criterio C4.5 evalúa una división, pero no determinan el orden de las decisiones ni cómo se recorre el árbol. Eso corresponde al procedimiento de construcción y predicción de la sección 3. Dos implementaciones pueden compartir todas las fórmulas y aun así diferir en los desempates, en las condiciones de parada o en el tratamiento de los valores faltantes. La subsección siguiente presenta Random Forest, que combina árboles de decisión con otro criterio de división.

2.2 Random Forest

Random Forest es un método de ensamble que combina muchos árboles de decisión para reducir la dependencia de una sola estructura. Un árbol aislado puede cambiar de forma ante variaciones pequeñas en el conjunto de entrenamiento, porque cada división se elige con los casos disponibles en ese nodo [2]. El ensamble reemplaza esa única estructura por un conjunto de árboles construidos con

perturbaciones aleatorias. Cada árbol se representa como $h(x, \Theta_k)$, donde Θ_k es un vector aleatorio que determina la muestra y los atributos disponibles en su construcción [1]. Los vectores Θ_k se generan de forma independiente y con la misma distribución. La predicción final no depende de un árbol individual, sino de la combinación de todos ellos.

La aleatoriedad del ensamble proviene de dos fuentes. La primera es el muestreo con reemplazo: cada árbol se entrena con una muestra del mismo tamaño que el conjunto original, de modo que algunos casos aparecen varias veces y otros quedan fuera [1]. La segunda es la selección de atributos: en cada división, el árbol evalúa solo un subconjunto aleatorio de los atributos disponibles, y no todos [1]. Esa restricción reduce la probabilidad de que un atributo dominante determine todas las divisiones y favorece que los árboles difieran entre sí. Cada rama se expande hasta que su hoja es pura o no se encuentra una división válida entre los atributos considerados; no se poda [1]. La profundidad máxima no se limita y la cantidad de atributos por división se fija como parámetro.

Para clasificar un caso nuevo, cada árbol emite un voto por una clase y el bosque asigna la clase más votada. Si B es la cantidad de árboles y $h_k(x)$ la clase que predice el árbol k para el caso x, la predicción del ensamble se define en la ecuación (11) [1].

$$\hat{y}(x) = \arg\max_{c} \sum_{k=1}^{B} \mathbf{1}[h_k(x) = c] \tag{11}$$

El voto es uniforme: todos los árboles pesan lo mismo, con independencia de su profundidad o de su desempeño individual. En scikit-learn, los empates se resuelven con el orden de las clases [3]. La votación convierte un conjunto de respuestas individuales en una sola etiqueta.

Para analizar el comportamiento del ensamble, Breiman define el margen de un caso como la diferencia entre la probabilidad de la clase correcta y la mayor probabilidad de una clase incorrecta, según la ecuación (12) [1].

$$mr(x, y) = P_{\Theta}(h(x, \Theta) = y) - \max_{j \neq y} P_{\Theta}(h(x, \Theta) = j) \tag{12}$$

En la ecuación, P_{Θ} indica la probabilidad sobre los vectores aleatorios que generan los árboles. Un margen positivo indica que, en promedio, los árboles votan por la clase correcta con mayor frecuencia que por cualquier otra; un margen negativo indica lo contrario. La fuerza del ensamble es el valor esperado $E_{X,Y}$ del margen sobre la distribución de los casos y las clases, ecuación (13).

$$s = E_{X,Y}[mr(X, Y)] \tag{13}$$

Cuanto mayor es la fuerza, más separación media existe entre la clase correcta y las alternativas, y la correlación media $\bar{\rho}$ mide cuánto se parecen los márgenes de dos árboles cualesquiera. La cota de la ecuación (14), para $s > 0$, relaciona el error de generalización PE^*, es decir, la probabilidad de que el margen sea

negativo, con esas dos cantidades: un error menor se asocia con árboles fuertes y poco correlacionados [1].

$$PE^* \leq \frac{\bar{\rho}(1 - s^2)}{s^2} \tag{14}$$

La cota explica de qué depende el error, pero no se usa para calcularlo; su utilidad es orientar el diseño del ensamble. Cuando la cantidad de árboles crece, el error de generalización converge a un límite, de modo que agregar árboles no lo incrementa de forma sistemática [1].

El muestreo con reemplazo deja fuera de cada árbol, en promedio, cerca de un tercio de los casos. Se los denomina casos fuera de la bolsa y no participan en la construcción de ese árbol. Si se predice cada caso solo con los árboles que no lo vieron, se obtiene una estimación del error de generalización sin separar un conjunto de prueba adicional [1]. La estimación es interna al entrenamiento y puede calcularse durante la construcción del bosque. En este trabajo se usa como referencia y no como sustituto de la partición de prueba independiente, que se reserva para comparar modelos.

Como configuración base, el Random Forest de scikit-learn usa cien árboles y crecimiento sin límite de profundidad [3]. En cada división evalúa la raíz cuadrada del número de atributos y usa el criterio de Gini; cada árbol se entrena con muestreo con reemplazo. La ecuación (11) describe el voto duro; la biblioteca, en cambio, promedia las probabilidades de los árboles y devuelve la clase con mayor probabilidad media [3]. Ambas reglas coinciden cuando cada árbol asigna probabilidad uno a su clase. El criterio de Gini evalúa una división con la impureza de los nodos resultantes, calculada como la probabilidad de clasificar mal un caso elegido al azar y etiquetado según la distribución del nodo [3]. La semilla y el número de hilos se fijan para que la ejecución sea reproducible. La descripción del algoritmo se presenta en la sección 3 y la de la implementación en la sección 4.

3 Breve descripción de los algoritmos desarrollados

Las secciones anteriores definieron los criterios y las fórmulas de ambos modelos. Esta sección describe cómo se aplican esos criterios en la construcción y la predicción. Primero se presenta el procedimiento de C4.5, con su pseudocódigo y las decisiones que lo completan. Después se describe el procedimiento de Random Forest y se cierra con una transición a la solución implementada.

3.1 C4.5

El árbol de C4.5 se construye de forma recursiva, dividiendo el conjunto de casos de cada nodo en subconjuntos más pequeños. La entrada del procedimiento es un conjunto de casos con sus clases, los atributos disponibles, los pesos que indican cuánto aporta cada caso y la profundidad del nodo. La salida es un nodo que puede ser una hoja, si se cumple una condición de parada, o un nodo de decisión

con un atributo, un umbral o un conjunto de ramas. En cada llamada se calculan los conteos ponderados por clase y se elige la clase mayoritaria, que se usará si el nodo termina como hoja. Luego se evalúan las divisiones posibles y se continúa por cada rama con el subconjunto correspondiente. El procedimiento se repite hasta que todas las ramas alcanzan una hoja [2]. El algoritmo 1 resume esa construcción.

El pseudocódigo muestra el orden de las decisiones, desde el cálculo de la clase mayoritaria hasta la creación recursiva de los hijos. El cuerpo se organiza en tres bloques: preparación del nodo, selección del atributo y expansión recursiva. Los pasos se numeran para ordenar la exposición y reflejan las decisiones adoptadas en este trabajo. La descripción se concentra en las decisiones que afectan el resultado y no en los detalles de programación. El procedimiento usa conteos ponderados en lugar de conteos simples, porque un caso con valores faltantes puede repartirse entre varias ramas con peso fraccionario.

Las paradas se evalúan en un orden fijo: primero la clase pura y el agotamiento de atributos, después la profundidad máxima y la masa mínima. La masa mínima se compara con la suma de pesos y no con la cantidad de filas, de modo que las copias fraccionarias no inflan la evidencia del nodo. Para cada atributo se evalúan todas las particiones candidatas y se conserva la de mayor tasa de ganancia ajustada. La comparación con el umbral de ganancia ajustada y con la tasa mínima es estricta: si el valor es menor, se detiene; la igualdad permite dividir. Los empates se resuelven por el orden de los atributos y, entre cortes numéricos del mismo atributo, por el menor umbral.

La predicción recorre el árbol desde la raíz hasta una hoja. Si el atributo de decisión tiene un valor conocido, el caso sigue la rama correspondiente: para un atributo numérico se compara con el umbral y para uno categórico se busca la categoría. Si el valor falta, el caso no se asigna a una sola rama: se combinan las distribuciones de todas las ramas con las proporciones aprendidas, como indica la ecuación (10). Si aparece una categoría que no se vio en ese nodo, se usan las frecuencias locales del nodo alcanzado. La clase final es la de mayor probabilidad, y los empates se resuelven con la primera clase observada en el entrenamiento. Las probabilidades son frecuencias ponderadas y no están calibradas.

El costo de construcción depende de la cantidad de atributos y de casos de cada nodo. Para un atributo numérico se ordenan sus valores observados, operación de costo $O(|D| \log |D|)$, y se evalúan $r-1$ cortes, donde r es la cantidad de valores distintos. Esa evaluación se repite en cada nodo y para cada atributo, de modo que el entrenamiento puede ser más costoso que el de un árbol que solo considera atributos categóricos. El crecimiento es codicioso: en cada nodo se elige la mejor división disponible y no se vuelve a evaluar esa decisión. El árbol resultante puede ser profundo y ajustarse en exceso al entrenamiento [2], y este trabajo no implementa pospoda ni poda de reglas para corregirlo. La única protección es la prepoda descrita, que puede detener el crecimiento antes de lo conveniente.

Algoritmo 1 Construcción recursiva de un árbol C4.5

Entrada: Casos D con clases, atributos A, pesos w, profundidad d (la raíz tiene $d = 0$), profundidad máxima, masa mínima, umbral de ganancia y tasa mínima.
Salida: Nodo del árbol.

Calcular la masa por clase y elegir la clase mayoritaria.
Inicializar un nodo con esa clase.
si hay una sola clase o A está vacío **entonces**
    devolver el nodo.
fin si
si d alcanzó el máximo o la masa es menor que el mínimo **entonces**
    devolver el nodo.
fin si
Inicializar el mejor candidato como vacío.
para cada atributo $a \in A$ **hacer**
    si a es numérico **entonces**
        Ordenar los valores observados y evaluar cada valor salvo el máximo.
    en otro caso
        Evaluar una rama por cada categoría conocida con masa positiva; se requieren al menos dos.
    fin si
    Calcular la tasa de ganancia ajustada por faltantes.
    Conservar la mayor tasa; desempatar por el orden de los atributos y, entre cortes numéricos, por el menor umbral.
fin para
si no hay candidato o la ganancia o la tasa no alcanzan sus mínimos **entonces**
    devolver el nodo.
fin si
Convertirlo en nodo de decisión y registrar el atributo, el umbral o las ramas y las proporciones.
Si el atributo es categórico, eliminarlo de A; si es numérico, conservarlo.
para cada rama **hacer**
    Formar el subconjunto conocido y repartir los faltantes con peso proporcional.
    Construir el hijo con el subconjunto, los atributos, los pesos y $d + 1$.
fin para
devolver el nodo.

3.2 Random Forest

El procedimiento de Random Forest construye B árboles, uno por cada muestra obtenida con reemplazo del conjunto de entrenamiento. Cada árbol se entrena con un subconjunto aleatorio de atributos en cada división y crece sin poda, como se describió en la sección 2.2. La cantidad de árboles y la cantidad de atributos por división son parámetros del método. La predicción combina los votos de los árboles o, en la implementación de scikit-learn, el promedio de sus probabilidades [3]. El resultado es un modelo que depende del conjunto de árboles y no de una sola estructura [1].

La diferencia con un árbol único está en la fuente de variabilidad. Un árbol construido con todos los atributos y todos los casos puede cambiar ante pequeñas

modificaciones de los datos; el bosque introduce aleatoriedad en la muestra y en los atributos de cada división para que los árboles no sean idénticos. Esa diversidad reduce la correlación entre los árboles y permite que el voto o el promedio combine respuestas distintas. La combinación no garantiza por sí sola un mejor desempeño: el resultado depende de la fuerza de los árboles y de su correlación, como indica la cota de la ecuación (14) [1]. Por eso la comparación con el árbol único debe hacerse con las mismas particiones y métricas, y no a partir de una expectativa general.

El costo de entrenamiento del bosque suele ser mayor que el de un árbol único, porque se construyen B árboles y cada uno recorre sus propios candidatos de división. La predicción también requiere recorrer todos los árboles y combinar sus respuestas. A cambio, la decisión deja de poder seguirse en un solo camino de la raíz a una hoja, porque el resultado depende del conjunto de árboles. La solución implementada organiza ambos modelos y resuelve los problemas técnicos que aparecen al integrarlos. La sección siguiente describe esa organización.

4 Descripción de la solución implementada

La sección anterior describió los procedimientos de ambos modelos. Esta sección presenta cómo se organiza el software que los implementa y qué problemas técnicos se resolvieron. Primero se describe la arquitectura general y el contrato común de predicción. Después se explican las decisiones de diseño y los problemas encontrados.

4.1 Arquitectura de software

La solución se organiza en tres capas: preparación de los datos, modelos y evaluación. La primera capa carga los conjuntos originales, verifica su integridad y construye las particiones. La segunda entrena el árbol de decisión C4.5 y el Random Forest. La tercera compara las predicciones con las clases reales y calcula las métricas. La Figura 1 resume el flujo entre esas capas y las dos ramas de modelado.

La preparación comienza con la verificación de integridad de los archivos que se leen desde disco, mediante un resumen criptográfico que detecta cambios en los bytes. Luego se decodifican las etiquetas numéricas de algunos conjuntos para que las clases tengan nombres legibles. La distinción entre atributos numéricos y categóricos se deduce del tipo de columna: los números se tratan como continuos, salvo los booleanos, y el texto y las categorías se tratan como discretos. La partición conserva la proporción de clases y reserva el veinte por ciento para prueba; usa una semilla fija de 42 para que sea reproducible. No se imputan valores faltantes ni se eliminan filas: la implementación de C4.5 los admite y la de Random Forest de scikit-learn los rechaza, como se explica más adelante.

El árbol C4.5 se implementa desde cero con funciones de métricas, una estructura de nodo y una construcción recursiva. Esa implementación conserva los atributos categóricos y elige una rama por categoría, de modo que no necesita

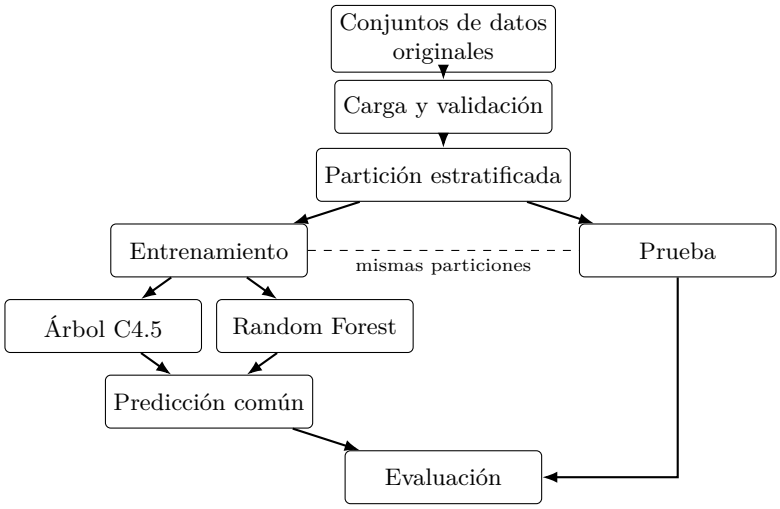


Fig. 1. Organización de la solución implementada.

una codificación previa. El Random Forest se toma de scikit-learn y se envuelve en un flujo de procesamiento que transforma las categorías en columnas binarias y conserva las columnas numéricas sin escalar. La codificación se ajusta solo con el conjunto de entrenamiento y se aplica después a los casos nuevos; una categoría no vista se representa con ceros. Esa codificación no se aplica a C4.5 porque cambiaría la semántica de la división: una columna binaria por categoría no equivale a una rama por categoría. La comparación se hace entre los modelos ya construidos, no entre sus representaciones internas.

Ambos modelos exponen una operación de predicción con el mismo contrato: recibe una tabla de datos, devuelve una columna con la clase predicha y conserva el índice y el orden de las filas. El contrato no unifica el entrenamiento ni las clases internas: cada modelo conserva su propia representación y su propio orden de clases. Esa separación reduce la dependencia entre las capas y permite comparar ambos modelos sobre las mismas filas de prueba. La evaluación usa las mismas particiones y una semilla fija en la separación y en el Random Forest. Los desempates del árbol son deterministas: se resuelven por el orden de los atributos y por la primera clase observada. El Random Forest se ejecuta con un solo hilo para que el resultado no dependa de la planificación de procesos. Las pruebas del árbol cargan las celdas de definición de su cuaderno, de modo que las definiciones probadas son las que usan los experimentos. Esa organización deja separadas la preparación, la construcción y la evaluación, y permite describir las decisiones de diseño a continuación.

5 Simulaciones y resultados

6 Conclusiones

References

1. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324
2. Liu, B.: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data. Data-Centric Systems and Applications, Springer, Berlin, Heidelberg, 2 edn. (2011). https://doi.org/10.1007/978-3-642-19460-3
3. scikit-learn developers: Random forests. `https://scikit-learn.org/stable/modules/ensemble.html#forest`, accessed 2026-10-15